VitalWatch

Technical Reference Documentation

Backend · Frontend · Integration · Deployment

1. Project Overview

VitalWatch is a post-surgery patient monitoring system built to run entirely on a Raspberry Pi 5 with a Hailo-8 AI accelerator. It connects bedside monitoring devices to a web dashboard, processes every vital sign reading through an AI inference engine, and alerts the care team in real time when a patient's condition changes.

1.1 Components

Component	Description
IoT Monitoring Device	Bedside oximeter and sensor unit. Reads heart rate, SpO_2 , respiration rate, and body temperature. Publishes readings via MQTT.
Raspberry Pi 5 (Edge Server)	Runs all backend services. Hosts the MQTT broker, FastAPI server, AI inference engine, and SQLite databases.
Hailo-8 AI Accelerator	Dedicated chip for running AI inference without loading the CPU. Model converted to HEF format for production.
VitalWatch Backend	Python/FastAPI application. Receives device readings, runs inference, manages alerts, serves the API and WebSocket.
VitalWatch Frontend	React web application. Real-time ward overview, patient detail with trend charts, AI assessment panel, admin tools.
Hospital PC	Any PC on the hospital network running a standard browser. No software installation required.

2. System Architecture

2.1 Data Flow

Every vital sign reading follows the same path from device to dashboard:

- Device publishes JSON to MQTT topic vitals/{device_id}
- MQTT subscriber receives the message and validates it via Pydantic

- VitalsService resolves the patient from the device assignment table
- Reading is saved to the database
- InferenceEngine predicts risk level, confidence, and explanation
- Assessment is saved to the database
- AlertService fires, suppresses, or resolves alerts based on the result
- PipelineResult is broadcast to all connected browsers via WebSocket
- Browser receives the push and fetches updated patient and alert data from the API

2.2 Dependency Rule

Imports only flow downward. Nothing lower in the stack ever imports from something above it. This makes each layer independently testable and replaceable.

Layer	Files	Responsibility
Entry points	main.py, mqtt_subscriber.py	Receive input, return output, no business logic
Routes	routes/*.py	Thin HTTP handlers — validate HTTP, call services, return responses
Services	services/*.py	All business logic and orchestration
Repositories	repositories/*.py	All SQL queries — services never write SQL
Core	core/database.py, core/inference.py	Database connections, schema, AI model
Models	models/*.py	Pydantic data shapes and validation — imported by all, imports nothing
Config	config.py	All constants — imported by all, imports nothing

3. Backend

3.1 Prerequisites

Requirement	Version	Notes
Python	3.11+	
Mosquitto	any	MQTT broker — sudo apt install mosquitto
AI model files	—	rf_model.pkl, scaler.pkl, feature_cols.pkl

3.2 Installation

```
cd backend
```

```
pip install fastapi uvicorn paho-mqtt pydantic bcrypt scikit-learn xgboost joblib
pandas numpy
```

3.3 Starting the System

```
sudo systemctl start mosquitto
uvicorn main:app --host 0.0.0.0 --port 8000
```

One command starts everything. The MQTT subscriber, WebSocket heartbeat, database initialisation, and AI model load all happen automatically on startup.

3.4 Configuration (config.py)

Every configurable constant lives in config.py. No values are hardcoded anywhere else. All settings can be overridden via environment variables with the same name.

Setting	Default	Description
MQTT_BROKER_HOST	localhost	Mosquitto host address
MQTT_BROKER_PORT	1883	Mosquitto port
DB_MONITORING	monitoring.db	Monitoring database file path
DB_USERS	users.db	Users and patients database file path
SUPPRESSION_MINUTES	{warning:30, critical:15}	Alert flood prevention windows
CORS_ORIGINS	http://localhost:5173	Allowed frontend origins
SESSION_MAX_AGE_HOURS	12	Login session lifetime

3.5 Database Schema

Two separate SQLite files. Neither is shared between systems.

users.db — Shared registry

Table	Contents
patients	Central patient registry — identity only: name, age, gender. Shared with the cancer detection system.
users	Staff accounts with bcrypt-hashed passwords and role assignments.
sessions	Active login sessions with expiry timestamps.

monitoring.db — Clinical data

Table	Contents
monitoring_enrollments	Links a patient to the ward — bed number, surgery info, assigned nurse.
devices	Known monitoring hardware devices, registered on first reading.
device_assignments	Maps device hardware IDs to patient IDs. One active assignment per device at a time.
readings	Every vital sign reading received from devices, stored permanently.
risk_assessments	Every AI inference result, linked to the reading that triggered it.
alerts	Every alert that fired, with status, severity, and suppression window.
alert_acknowledgements	Audit trail — who took what action on which alert and when.

3.6 API Endpoints

Authentication — /api/auth

Method	Endpoint	Description
POST	/api/auth/login	Validate credentials, set session cookie
GET	/api/auth/me	Return current user from session
POST	/api/auth/logout	Revoke session, clear cookie
POST	/api/auth/users	Create staff account (admin only)

Patients — /api/patients

Method	Endpoint	Description
POST	/api/patients	Register patient in central registry
POST	/api/monitoring/enroll	Enroll registered patient in monitoring
GET	/api/patients	Ward overview — all active patients with latest status
GET	/api/patients/{id}/status	Full status for one patient
GET	/api/patients/{id}/readings?minutes=180	Reading history for trend charts

Alerts — /api/alerts

Method	Endpoint	Description
GET	/api/alerts/count	Unacknowledged count and ward status
GET	/api/alerts	Alert feed with optional status/severity/patient_id filters
GET	/api/alerts/patient/{id}	Alert history for one patient

Method	Endpoint	Description
POST	/api/alerts/{id}/acknowledge	Nurse acknowledges an alert
POST	/api/alerts/{id}/escalate	Nurse escalates to physician
POST	/api/alerts/{id}/resolve	Manually resolve an alert

Devices — /api/devices

Method	Endpoint	Description
GET	/api/devices	All known devices with current assignment status
POST	/api/devices/assign	Assign device to patient
POST	/api/devices/unassign	Remove device assignment

WebSocket — /ws/ward

A persistent connection that the browser holds for the entire session. The server pushes updates immediately when a new reading arrives. Message types:

Direction	Type	Description
Server → Browser	patient_update	New reading processed — refresh patient and alert data
Server → Browser	ping	Heartbeat every 25 seconds
Browser → Server	pong	Heartbeat acknowledgement

3.7 Device Integration

Devices do not send a patient_id. They send their own hardware ID. The backend resolves which patient the reading belongs to via the device_assignments table.

MQTT topic: vitals/{device_id}

Required JSON fields:

Field	Type	Validation
device_id	String	Required
heart_rate	Number	20 – 300 bpm
spo2	Number	50 – 100 %
respiration_rate	Number	1 – 60 brpm
body_temperature	Number	90 – 110 °F

Field	Type	Validation
read_at	String (optional)	ISO 8601. Defaults to server time if omitted.

3.8 Alert Rules

Four rules evaluated on every incoming reading:

- Stable reading — auto-resolves all open alerts for the patient
- Warning or Critical — fires a new alert unless the suppression window is active for that severity
- Worsening severity — fires immediately regardless of suppression (e.g. patient moves from Warning to Critical)
- Suppression reminder — after the window expires, a new alert fires if the patient is still unwell

3.9 AI Model

Four input vitals, 23 engineered features, three output classes (Stable, Warning, Critical).

Feature Group	Count	Features
Raw vitals	4	heart_rate, spo2, respiration_rate, body_temperature
Rolling mean	4	Average of last 3 readings per vital
Delta	4	Change from previous reading per vital
Rolling std dev	4	Fluctuation over last 5 readings per vital
Deviation from normal	4	Distance from midpoint of normal range per vital
Interaction features	3	SpO2×RR, HR×Temp, SpO2/RR

Random Forest was selected over XGBoost for its higher Critical Recall (77.0% vs 67.8%). In a clinical context, missing a critical patient is more dangerous than a false alarm.

4. Frontend

4.1 Prerequisites

Requirement	Notes
Node.js 18+	Download LTS from nodejs.org — includes npm

Requirement	Notes
Backend running on port 8000	Required for login and data

4.2 Development Setup

```
cd vitalwatch-frontend
npm install
npm run dev
```

Open http://localhost:5173. The dev server proxies all /api calls to http://localhost:8000 automatically.

4.3 Production Build

```
npm run build
```

Outputs plain HTML, CSS, and JavaScript to the dist/ folder. Copy into the backend frontend/ directory. The Pi does not need Node.js installed for production.

4.4 Demo Accounts

Email	Password	Role	Landing Page
nurse@hospital.org	password123	Nurse	Ward Overview
doctor@hospital.org	password123	Doctor	Ward Overview
radiologist@hospital.org	password123	Radiologist	Cancer Detection
admin@hospital.org	password123	Admin	Admin Panel

4.5 Real-Time Architecture

One WebSocket connection is held per browser session by the WardSocketProvider component, which wraps all monitoring routes. Both the navbar and the ward overview consume the same context — no duplicate connections.

On every server push, the frontend makes one API call to refresh the full patient and alert data. On disconnect, automatic reconnection starts at 2 seconds and backs off exponentially to a maximum of 30 seconds.

4.6 Theme System

Light mode is the default. The theme toggle in the navbar switches to dark mode. The preference persists in localStorage.

Every component calls const C = useColors() which returns the appropriate color palette. No hardcoded hex values exist in any component. Switching themes is a single state change that updates the entire UI simultaneously.

4.7 Font System

Font	Role	Used For
Manrope	Headlines	Patient names, vital numbers, screen titles, risk level labels
Inter	Body	Descriptions, alert text, general UI, button labels
IBM Plex Sans	Labels	ALL CAPS section headings, unit strings, timestamps, small tags

5. Deployment

5.1 Startup Order

- sudo systemctl start mosquitto
- uvicorn main:app --host 0.0.0.0 --port 8000
- Open browser on hospital PC and navigate to the Pi IP address

5.2 Recommended Pi Setup

Give the Pi a static IP so its address never changes:

```
# Edit /etc/dhcpcd.conf
interface eth0
static ip_address=192.168.1.50/24
static routers=192.168.1.1
```

Enable Mosquitto on boot so it starts automatically:

```
sudo systemctl enable mosquitto
```

5.3 Nginx Configuration (clean URL)

Place Nginx in front to serve everything on port 80 so nurses type http://vitalwatch instead of http://192.168.1.50:8000.

```
server {
    listen 80;
    location /api/ { proxy_pass http://localhost:8000; }
    location /ws/  { proxy_pass http://localhost:8000; proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection upgrade; }
```

```
    location /     { proxy_pass http://localhost:8000; }
}
```

5.4 URL Configuration (hosts file)

To use http://vitalwatch in the browser, add one line to the hosts file on each hospital PC:

```
# Windows: C:\Windows\System32\drivers\etc\hosts
# Linux/Mac: /etc/hosts
192.168.1.50    vitalwatch
```

6. Complete Patient Workflow

Every patient goes through this workflow before monitoring begins.

Step	Who	How	Notes
1. Register patient	Admin	Admin Panel → Patient Registration tab, or POST /api/patients	Captures name, age, gender in the central registry
2. Enroll in monitoring	Admin	Same form on submit, or POST /api/monitoring/enroll	Captures bed number, surgery type, surgery date
3. Assign device	Admin or Nurse	Admin Panel → Device Assignment tab, or POST /api/devices/assign	Links the hardware device ID to the patient
4. Attach device	Nurse	Physical attachment of device to patient	Device starts publishing readings immediately
5. Monitor	Nurse/Doctor	Ward Overview and Patient Detail screens	Dashboard updates in real time via WebSocket
6. Discharge	Admin or Nurse	Unassign device via Admin Panel or POST /api/devices/unassign	Device is freed for the next patient

7. Troubleshooting

Symptom	Likely Cause	Fix
Ward overview shows no patients	No patients registered and enrolled	Use Admin Panel to register and enroll at least one patient, then assign a device

Symptom	Likely Cause	Fix
Patient cards show no readings	Device not assigned or MQTT subscriber not running	Check backend terminal for [mqtt] log lines. Assign device via Admin Panel.
Login fails with network error	Backend not running	Start uvicorn and refresh
Charts show no data for 30/60 min	Not enough recent readings accumulated	Start simulator or connect device and wait a few minutes
Connection banner shows "Connection lost"	WebSocket endpoint unreachable	Check backend is running. Check browser console for error details.
"no such table: patients" error	init_db() not called before first use	Restart the backend — init_db() runs on startup and creates all tables
Readings show but wrong patient	Device assigned to wrong patient	Unassign and reassign the device via Admin Panel

End of Document